



1. Problem Title & Link

Title: LeetCode 1652 – Defuse the Bomb

Link: <https://leetcode.com/problems/defuse-the-bomb/>

2. Problem Statement (Short Summary)

You are given a circular array code and an integer k.

For every index:

- If $k > 0$, replace each element with the sum of the next k elements.
- If $k < 0$, replace each element with the sum of the previous $|k|$ elements.
- If $k == 0$, replace every element with 0.

Return the decrypted array.

3. Examples (Input → Output)

Example 1

Input

code = [5,7,1,4]

k = 3

Output

[12,10,16,13]

Explanation

- $5 \rightarrow 7+1+4 = 12$
- $7 \rightarrow 1+4+5 = 10$
- $1 \rightarrow 4+5+7 = 16$
- $4 \rightarrow 5+7+1 = 13$

Example 2

Input

code = [1,2,3,4]

k = 0

Output

[0,0,0,0]

Example 3

Input

code = [2,4,9,3]

k = -2

Output



[12,5,6,13]

4. Constraints

- $1 \leq \text{code.length} \leq 100$
- $1 \leq \text{code}[i] \leq 100$
- $-(n-1) \leq k \leq n-1$
- Array is circular.

5. Core Concept (Pattern / Topic)

- Sliding Window
- Circular Array
- Simulation

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every index:

- Visit the next/previous $|k|$ elements.
- Add them one by one.

Time Complexity:

$O(n \times |k|)$

Since we repeat the same additions many times, there is unnecessary work.

Optimized Thinking

Notice that consecutive windows overlap.

Instead of recomputing every sum:

- Compute the first window sum.
- Slide the window by:
 - Removing one element.
 - Adding one new element.

Since the array is circular, use modulo $\% n$ to wrap around.

Intuition

If $k > 0$

Window

[i+1 ... i+k]



Slide Right

Remove leftmost

Add next element

If $k < 0$

Window

$[i-k \dots i-1]$



Slide Right

Remove oldest

Add current index

Why This Works

Every window differs from the previous one by only one element.

Sliding Window lets us update the sum in constant time instead of recomputing it.

7. Visual / Intuition Diagram (ASCII)

```
Example
code = [5,7,1,4]
k = 3
Index 0
```

```
      [7 1 4]
5 -> (sum =12)
```



Slide

```
      [1 4 5]
7 -> (sum =10)
```



Slide



```
[4 5 7]
1 -> (sum =16)
```

8. Pseudocode (Language Independent)

```
if k == 0
    return all zeros

initialize first window

for every index

    answer[i] = currentSum

    remove outgoing element

    add incoming element

return answer
```

9. Code Implementation

Python

```
class Solution:
    def decrypt(self, code: list[int], k: int) -> list[int]:

        n = len(code)

        if k == 0:
            return [0] * n

        ans = [0] * n

        if k > 0:

            window = sum(code[1:k + 1])

            for i in range(n):
                ans[i] = window
                window -= code[(i + 1) % n]
                window += code[(i + k + 1) % n]

        else:

            k = -k
            window = sum(code[n - k:])

            for i in range(n):
                ans[i] = window
```



```
        window -= code[(i - k + n) % n]
        window += code[i]

    return ans
```

Java

```
class Solution {

    public int[] decrypt(int[] code, int k) {

        int n = code.length;
        int[] ans = new int[n];

        if (k == 0)
            return ans;

        if (k > 0) {

            int window = 0;

            for (int i = 1; i <= k; i++)
                window += code[i];

            for (int i = 0; i < n; i++) {
                ans[i] = window;
                window -= code[(i + 1) % n];
                window += code[(i + k + 1) % n];
            }

        } else {

            k = -k;

            int window = 0;

            for (int i = n - k; i < n; i++)
                window += code[i];

            for (int i = 0; i < n; i++) {
                ans[i] = window;
                window -= code[(i - k + n) % n];
                window += code[i];
            }

        }

        return ans;
    }
}
```



10. Time & Space Complexity

Time Complexity

$O(n)$

- Initial window computation: $O(k)$
- Sliding through the array: $O(n)$
- Since $k \leq n$, overall complexity is $O(n)$.

Space Complexity

$O(n)$

For the output array.

11. Common Mistakes / Edge Cases

- Forgetting $k == 0$
- Missing circular indexing ($\% n$)
- Wrong initial window
- Incorrect outgoing/incoming element while sliding
- Forgetting to convert negative k into $|k|$

12. Detailed Dry Run

Example

code = [5,7,1,4]

$k = 3$

Initial window

$7 + 1 + 4 = 12$

i	Window	Sum	Answer
0	7,1,4	12	12
1	1,4,5	10	10
2	4,5,7	16	16
3	5,7,1	13	13

Final Output

[12,10,16,13]

13. Common Use Cases (Real-Life / Interview)

- Circular sensor readings
- Ring buffers
- Network token ring systems



- Rotating queues
- Fixed-size moving averages
- Circular scheduling

14. Common Traps (Important!)

- Off-by-one errors
- Wrong modulo calculation
- Mixing previous and next windows
- Computing the initial window incorrectly
- Recomputing the window every iteration instead of sliding

15. Builds To (Related LeetCode Problems)

- LC 643 – Maximum Average Subarray I
- LC 2090 – K Radius Subarray Averages
- LC 239 – Sliding Window Maximum
- LC 1423 – Maximum Points You Can Obtain from Cards
- LC 2134 – Minimum Swaps to Group All 1's Together II

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n \times k)$	k	
Prefix Sum on Doubled Array	$O(n)$	$O(n)$	Easy handling of circularity
Sliding Window	$O(n)$	$O(n)$	Best interview solution

17. Why This Solution Works (Short Intuition)

Each answer depends on a fixed-size window of neighboring elements. Since consecutive windows overlap heavily, we reuse the previous window sum by removing one element and adding one new element. Circular indexing using modulo ensures the window wraps around correctly.

18. Variations / Follow-Up Questions

1. Solve using a doubled array and prefix sums.
2. Return the product instead of the sum.
3. Handle variable window sizes for each index.
4. Support online updates to the array.
5. Extend the approach to a 2D circular matrix.



6.